Ders 14 Genişletilmiş Notu: Proje Gösterimleri (Son Hafta)

Ders 14 Genişletilmiş Notu: Proje Gösterimleri (Son Hafta)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kaydın Kapsamı
- 2. Simpson 1/3 Sınaması ve Girdi Soruları
- 3. Gauss-Seidel: Maksimum Köşegen
- 4. Gregory-Newton Enterpolasyonu
- 5. Gauss Eliminasyonu ve Başlangıç Değeri
- 6. Matrisin Tersi
- 7. Newton-Raphson
- 8. Merkezi Farkla Sayısal Türev
- 9. Bisection ve Kökü Yerine Koyarak Sınama
- 10. Üç Bilinmeyenli Gauss-Seidel ve Trapez
- 11. Ortak Fonksiyona Parametre Aktarımı

1. Kaydın Kapsamı

Bu kayıtta yeni konu işlenmedi. Hoca, dönem boyunca görülen yöntemlerin kodlarını öğrencilerin programlarında çalıştırarak sınadı. Bu not yalnız o sırada kullanılan sınama örneklerini ve hocanın verdiği akademik düzeltmeleri içerir.

Hoca sınamada şu düzeni kullandı: aynı girdiler iki programa birden veriliyor, sonuçlar karşılaştırılıyor. Ayrıca çoğu yerde sonucu elle ya da hesap makinesiyle kontrol etti.

2. Simpson 1/3 Sınaması ve Girdi Soruları

Hoca integral sınamasında şu örneği verdi: $f(x) = 2x^5 - 3x^2 + 3$, aralık $[1, 4]$, aralık sayısı $n = 6$.

Adım uzunluğu $h = (4 - 1)/6 = 0.5$ olur (ara adım eklendi). Simpson 1/3 formülü:

$$I \approx \frac{h}{3}\left[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + 4f(x_5) + f(x_6)\right]$$

Derste sonuç 1311 olarak okundu ve doğru kabul edildi.

> **Ders sonrası not:** Python ile hesaplandı: bu örneğin Simpson 1/3 değeri $n = 6$ için 1311.625, integralin tam değeri ise 1311'dir. Altyazıda okunan 1311 sayısının ondalık kısmı duyulmuyor.

2.1 Sıfır Katsayılı Kuvvetler ve Terim Sayısı

İlk denemede bir karışıklık çıktı. Hoca polinomun kuvvetlerini sırayla söyledi ve x^4 ile x^3 için katsayı olarak 0 verdi. Program ise önce terim sayısını sormuştu. Hoca durumu şöyle anlattı:

bu polinomda 3. ve 4. kuvvetler yok, bu yüzden onları terim sayısına katmadı. Terim sayısı 3 alındığında program yalnız üç terim beklediği için sıfır katsayılı kuvvetlerin de girilmesi girdiyi bozdu. Terimler yalnız var olan kuvvetlerle yeniden girilince doğru sonuç alındı.

2.2 Soruyu Doğru Sormak

Gauss-Seidel sınamasında bir programın "denklemin derecesi" diye sorduğunu gören hoca, bunun aslında kaç denklem olduğunu sorduğunu anlayınca şöyle dedi: "Denklemin derecesi diye sorarsan bambaşka şey anlaşılır." Burada istenen değer denklem sayısıdır.

Benzer bir uyarı Newton-Raphson için de geldi. Bir program polinomun bulunduğu aralığın en küçük ve en büyük sayısını soruyordu. Hoca, Newton'da aralık olmadığını söyledi.

3. Gauss-Seidel: Maksimum Köşegen

Hocanın iki bilinmeyenli örneği, başlangıç değerleri $(1, 1)$ ile:

$$2x_1 + 5x_2 = 16, \qquad 3x_1 + x_2 = 11$$

Hata payı 0.01, iterasyon sayısı 100 girildi. Bu sırayla verildiğinde sınanan programlar sonuç üretemedi. Hoca da "demek ki köşegeni kendin maksimuma getirmiyorsun" anlamında yorumladı. Ardından denklemlerin sırasını kendisi değiştirip köşegeni en büyük olacak şekilde verdi:

$$3x_1 + x_2 = 11, \qquad 2x_1 + 5x_2 = 16$$

Bu düzenle doğru sonuç $x_1 = 3,\ x_2 = 2$ çıktı. Python ile hesaplanan iterasyonlar (ara adım eklendi):

İterasyon	x_1	x_2
1	3.33333	1.86667
2	3.04444	1.98222
3	3.00593	1.99763
4	3.00079	1.99968

Hazır düzenle 3 ve 2 bulan bir programın ilk sıralamada sonuç üretememesini hoca, maksimum köşegen kontrolünün yanlış çalışmasına bağladı. Aynı sistemi hazır düzenle de çözemeyen başka bir programda ise Gauss-Seidel kısmının hatalı olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Her satırda köşegen elemanının mutlak değeri, o satırın diğer elemanlarının mutlak değerleri toplamından büyükse Gauss-Seidel yakınsar. Bu yeter koşuldur, gerekli değildir. Python ile hesaplandı: ilk sıralamayla $(1, 1)$'den başlayınca iterasyonlar $(5.5, -5.5)$, $(21.75, -54.25)$, $(143.625, -419.875)$ biçiminde ıraksar. Satır değiştirmenin nedeni budur.

4. Gregory-Newton Enterpolasyonu

Hoca beş veri verdi ve $x = 1895$ için değeri sordu. Bir program x değerlerini tek tek alıyordu, diğeri başlangıç değeri ve artımdan kendisi üretiyordu. İki yol da aynı sonucu verdi.

x	1891	1901	1911	1921	1931
y	46	66	81	93	101

İleri farklar (ara adım eklendi): $\Delta y_0 = 20$, $\Delta^2 y_0 = -5$, $\Delta^3 y_0 = 2$, $\Delta^4 y_0 = -3$. Adım $h = 10$, dolayısıyla $s = (1895 - 1891)/10 = 0.4$ olur.

$$y(1895) = 46 + 0.4(20) + \frac{0.4(-0.6)}{2}(-5) + \frac{0.4(-0.6)(-1.6)}{6}(2) + \frac{0.4(-0.6)(-1.6)(-2.6)}{24}(-3)$$

$$y(1895) = 46 + 8 + 0.6 + 0.128 + 0.1248 = 54.8528$$

Derste okunan 54.85 ve 54.8528 ile uyumludur; python ile hesaplandı.

İlerleyen gösterimde ikinci bir örnek verildi: $x = 0, 1, \ldots, 6$ için $y = -4, -2, 14, 62, 160, 326, 578$ ve istenen $x = 4$. Sonuç 160 çıktı, hoca doğru dedi. Bu değer tablodaki $x = 4$ değeridir.

> **Ders sonrası not:** Python ile hesaplandı: bu tablonun üçüncü farkları sabit 18'dir ve veri $3x^3 - 2x^2 + x - 4$ polinomuna tam oturur. Tablo noktasında sorulduğu için bu örnek, enterpolasyonun ara değerdeki hesabını sınamaz.

5. Gauss Eliminasyonu ve Başlangıç Değeri

Bir programın Gauss-Seidel sırasında başlangıç değeri istemediği görülünce kısa bir tartışma oldu.

Soru: Gauss-Seidel'de yalnız hata payı ve iterasyon sayısını almak yeterli olur mu?

Cevap: Hayır. Gauss-Seidel başlangıç değeri olmadan çözülemez. Eliminasyon ise çözülür, çünkü matrisi üst üçgen hâle getirir, sonra geriden başlayarak çözer. Bunun için bir başlangıç değerine ihtiyaç yoktur.

> **Hocanın vurgusu:** "Seidel'ı başlangıç değeri almadan çözemezsin. Eliminasyonu çözersin."

Hocanın eliminasyon örneği:

$$\begin{aligned} x_1 + x_2 + x_3 &= 25 \\ 5x_1 + 3x_2 + 2x_3 &= 0 \\ x_2 - x_3 &= 6 \end{aligned}$$

İşlemler (ara adım eklendi):

1. $R_2 \leftarrow R_2 - 5R_1$ ile ikinci satır $-2x_2 - 3x_3 = -125$ olur.
2. $R_3 \leftarrow R_3 + \frac{1}{2}R_2$ ile üçüncü satır $-2.5x_3 = -56.5$ olur. Matris üst üçgen hâle geldi.
3. Geriden çözüm: $x_3 = 22.6$.
4. $-2x_2 - 3(22.6) = -125$ eşitliğinden $x_2 = 28.6$.
5. $x_1 + 28.6 + 22.6 = 25$ eşitliğinden $x_1 = -26.2$.

Sonuç $x_1 = -26.2$, $x_2 = 28.6$, $x_3 = 22.6$ olarak derste de okundu ve doğru kabul edildi; python ile hesaplandı. Kaydın başka yerlerinde bu sonuç kısaca "26, 28, 22" diye geçer, tam değerler yukarıdakilerdir.

6. Matrisin Tersi

Bir programda matris tekil (singular) çıkınca neden olarak girişin yanlış yazılmış olabileceği konuşuldu. Hoca da aynı girdileri iki programa birden vermesinin nedeninin bu tür yanlışları görmek olduğunu söyledi.

Hocanın üç boyutlu örneği:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 2 & 9 \end{bmatrix}, \qquad \det A = -36$$

Python ile hesaplanan ters matris:

$$A^{-1} \approx \begin{bmatrix} -0.9167 & 0.3333 & 0.0833 \\ -0.1667 & 0.3333 & -0.1667 \\ 0.75 & -0.3333 & 0.0833 \end{bmatrix}$$

Ders sonrası not: Hoca ekrandaki sonucu okurken son eleman altyazıda "eksi sıfır seksen üç" diye geçiyor. Python ile hesaplandı: A^{-1}'in son elemanı $+0.0833$'tür. Okunan diğer değerler (-0.92, 0.33, 0.08, -0.17, 0.75, -0.33) hesapla uyumludur.

Hocanın dört boyutlu örneği:

$$B = \begin{bmatrix} 1 & 3 & 5 & 9 \\ 1 & 3 & 1 & 7 \\ 4 & 3 & 9 & 7 \\ 5 & 2 & 0 & 9 \end{bmatrix}, \qquad \det B = -376$$

Python ile hesaplanan ters matris:

$$B^{-1} \approx \begin{bmatrix} -0.277 & 0.043 & 0.149 & 0.128 \\ -0.625 & 0.875 & 0.250 & -0.250 \\ 0.104 & -0.141 & 0.069 & -0.048 \\ 0.293 & -0.218 & -0.138 & 0.096 \end{bmatrix}$$

7. Newton-Raphson

Hocanın örneği: $f(x) = x^3 + x^2 - 9x - 9$, başlangıç değeri $x_0 = -5$, $\varepsilon = 0.001$.

Soru: Newton-Raphson'da başlangıç ve bitiş değerine ihtiyaç var mı?

Cevap: Hayır, sadece başlangıç değerine ihtiyaç var.

Türev $f'(x) = 3x^2 + 2x - 9$ ve yineleme $x_{k+1} = x_k - f(x_k)/f'(x_k)$ ile (ara adım eklendi, python ile hesaplandı):

k	x_k	$\lvert x_k - x_{k-1}\rvert$
1	-3.857143	1.142857
2	-3.255639	0.601504
3	-3.034153	0.221486
4	-3.000750	0.033403
5	-3.000000	0.000750

Sonuç -3 çıktı. Hoca bunun "köklerden biri" olduğunu vurguladı. Polinomun kökleri -3, -1 ve 3'tür; python ile hesaplandı.

Hocanın sonradan kullandığı $x^2 - 4x - 5$ için de "kök zaten -1 ile 5, ikisinden birini bulacak" dedi. Bir başlangıç değeri, polinomun yalnız bir kökünü verir.

8. Merkezi Farkla Sayısal Türev

Hocanın ana örneği $f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$, nokta $x = 1$, $h = 0.25$. Merkezi farkı seçme nedenini "kontrolü kolay" diye açıkladı.

Hoca sonucu analitik türevle kontrol etti: $f'(x) = -0.4x^3 - 0.45x^2 - x - 0.25$, dolayısıyla $f'(1) = -0.4 - 0.45 - 1 - 0.25 = -2.1$.

Merkezi fark formülü:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Sayısal hesap (ara adım eklendi): $f(1.25) = -0.430859$, $f(0.75) = 0.636328$ ve sonuç $(-0.430859 - 0.636328)/0.5 = -2.134375$. Derste programların verdiği değer "-2.1" diye okundu ve doğru kabul edildi.

> **Ders sonrası not:** Python ile hesaplandı: -2.1 analitik türevin değeri, $h = 0.25$ için merkezi farkın değeri ise -2.134375'tir. Bir programda h önceden tanımlıydı, hoca "sonuç değiştirmez" dedi. Doğrusu h sonucu değiştirir. Bu polinom için merkezi fark tam olarak $f'(x) + \frac{h^2}{6} f'''(x)$ verir ve h küçüldükçe -2.1'e yaklaşır.

8.1 Katsayı Okuma Döngüsü

Bir programda ilk katsayı olan -0.1 girilince geri kalan girdiler sorulmadan program türev noktasına atlıyordu. Hoca koda birlikte baktı. Katsayılar büyük dereceden küçüğe doğru alınıyordu ve döngünün mantığını adım adım anlattı:

- i, d'den başlar. Örnekte $d = 4$ olduğu için i sırayla 4, 3, 2, 1, 0 olur.
- Her adımda i azalır.
- Döngü $i \geq 0$ olduğu sürece döner. i sıfırdan küçük olunca çıkılır.

Hoca burada `for` yerine `while` kullanmanın da uygun olacağını söyledi. Aynı incelemede döngünün gövdesinin erken bittiği, indis ve derece değişkenlerinin (i ile d) karıştığı, açılıp kapanan parantezlerin de kontrol edilmesi gerektiği konuşuldu. Hocanın genel uyarısı şuydu: "Bazen takıldığımız şeyi göremeyiz."

8.2 Hatanın Basit Fonksiyonlarla Daraltılması

Hoca hatayı bulmak için fonksiyonları giderek basitleştirip her sonucu analitik türevle karşılaştırdı. Hepsinde $x = 1$, $h = 0.25$ idi.

Fonksiyon	Analitik türev $f'(1)$	Merkezi fark (python)
$x^3 - x^2 - x - 1$	$3 - 2 - 1 = 0$	0.0625
$x^2 - 4$	2	2
$x^3 - 4x^2 + 1$	$3 - 8 = -5$	-4.9375
$x^4 + x^3 + x^2 + x + 1$	$4 + 3 + 2 + 1 = 10$	10.3125

İlk satırda program 0.06 verince hoca, 0 çıkması gerektiğini söyleyip programda hata olduğunu belirtti.

> **Ders sonrası not:** Hoca 0 çıkması gerektiğini, 0.06'nın hata olduğunu söyledi. Python ile hesaplandı: $h = 0.25$ için merkezi farkın doğru değeri 0.0625'tir, çünkü $\frac{h^2}{6} f'''(1) = \frac{0.0625}{6} \cdot 6 = 0.0625$ olur. 0 değeri analitik türevdir, ekrandaki 0.06 bu satırda hata göstermez.

Tam sayılı katsayılarla girdi sorunu görülmeyince hoca teşhisini söyledi: "Katsayıları 0.1 veriyoruz. Sen onları integer alıyorsun galiba. Hatan oradan." Sonraki gösterimlerde de aynı konuya döndü ve katsayı değişkeninin integer tanımlanmamış olmasını istedi. Bir öğrencinin önceki Simpson ve trapez hatasının sebebi de değişkeni `double` yerine `int` tanımlamaktı.

> **Hocanın vurgusu:** Ondalıklı katsayı alan değişkenler integer tanımlanmamalıdır.

> **Ek bilgi (derste anlatılmadı):** `scanf("%d", ...)` girişte `-0.1` görünce noktada durur, `.1` kısmı girişte kalır. Sonraki `%d` okumaları o noktada başarısız olur ve kullanıcıyı beklemeden geçer; gcc ile derlenip çalıştırıldığında bu davranış görüldü. Programın sorulmadan türev noktasına atlaması büyük olasılıkla bu yüzdendir. Aşağıdaki program hocanın tarif ettiği döngüyle katsayıları `double` okur ve merkezi farkı hesaplar. `gcc -Wall` ile uyarısız derlendi; ana örnek için `-2.134375` verdi.

```c
#include <stdio.h>

#define MAKS 50
```

```c
double polinom(const double a[], int d, double x)
{
    double s = 0.0;
    int i = d;

    while (i >= 0) {
        s = s * x + a[i];
        i--;
    }
    return s;
}

int main(void)
{
    double a[MAKS], x, h;
    int d, i;

    if (scanf("%d", &d) != 1 || d < 0 || d >= MAKS)
        return 1;
    for (i = d; i >= 0; i--)
        if (scanf("%lf", &a[i]) != 1)
            return 1;
    if (scanf("%lf %lf", &x, &h) != 2)
        return 1;

    printf("%.6f\n", (polinom(a, d, x + h) - polinom(a, d, x - h)) / (2 * h));
    return 0;
}
```

9. Bisection ve Kökü Yerine Koyarak Sınama

9.1 Birinci Örnek: $[4, 7]$ Aralığı

Hocanın örneği $f(x) = x^2 - 4x - 5$, aralık $[4, 7]$, $\varepsilon = 0.01$. Bir programın sorduğu iki durma koşulu için hoca "sen istediğini seç, yeter ki sonuç bul" dedi. Sonuç 4.9990 çıktı, kök 5 kabul edildi.

Python ile hesaplanan orta noktalar (ara adım eklendi): 5.5, 4.75, 5.125, 4.9375, 5.03125, 4.984375, 5.0078125, 4.99609375, 5.001953125, 4.9990234375. Onuncu adımda $|f(m)| \approx 0.00586 < 0.01$ olduğu için $|f(m)| < \varepsilon$ ölçütü burada durur. Derste okunan 4.9990 ile uyumludur.

9.2 İkinci Örnek: Kök Olmayan Sonuç

Hoca $f(x) = x^3 + x^2 - 9x - 9$ fonksiyonunu $[-5, -2]$ aralığında verdi. İki program da -3.5 buldu. Katsayılar yeniden girildi, sonuç değişmedi. Hoca bu sefer -3.5'i yerine koyarak hesapladı:

$$f(-3.5) = -42.875 + 12.25 + 31.5 - 9 = -8.125$$

Yerine koyunca sıfır çıkmadı. Hoca iki programın yine de aynı değeri vermesini enteresan buldu.

> **Hocanın vurgusu:** Bulunan kök yerine konduğunda tam sıfır çıkmasa bile 0.01 gibi küçük bir değer çıkmalıdır. Hoca hesabı öğrencilere de yaptırmak istedi: "Niye hazıra konuyorsunuz?"
>
> **Ders sonrası not:** Python ile hesaplandı: $f(-5) = -64$, $f(-2) = 5$ ve -3.5 bu aralığın ilk orta noktasıdır. Yöntem devam ettirilince orta noktalar -2.75, -3.125, -2.9375, -3.03125, ... ile -3 köküne gider. Programların durması büyük olasılıkla ilk adımda kalan bir döngü ya da durma koşulu hatasından kaynaklanıyordu.

9.3 Üçüncü Örnek: $[-3, 0]$ Aralığı

Hoca fonksiyonu $x^2 - 4x - 5$ ile değiştirip aralığı $[-3, 0]$ verdi. Sonuç -0.9843 ve -0.98437 okundu. Hoca bunu "-1'e yakın, -1 sayılır" diye kabul etti.

Python ile hesaplanan orta noktalar (ara adım eklendi): -1.5, -0.75, -1.125, -0.9375, -1.03125, -0.984375. Okunan değer altıncı orta noktadır.

10. Üç Bilinmeyenli Gauss-Seidel ve Trapez

Hocanın üç bilinmeyenli örneği, başlangıç değerleri $(0, 0, 0)$, hata payı 0.001:

$$\begin{aligned} 5x_1 - 2x_2 + 3x_3 &= -1 \\ -3x_1 + 9x_2 + x_3 &= 2 \\ 2x_1 - x_2 - 7x_3 &= 3 \end{aligned}$$

Python ile hesaplanan iterasyonlar:

İterasyon	x_1	x_2	x_3	$\max\lvert\Delta x\rvert$
1	-0.2	0.15556	-0.50794	0.50794
2	0.16698	0.33432	-0.42862	0.36698
3	0.19090	0.33348	-0.42167	0.02392
4	0.18639	0.33121	-0.42263	0.00451
5	0.18606	0.33120	-0.42273	0.00033

Hoca ilk iterasyonu (-0.2, 0.15, -0.5) ve sonucu (0.18, 0.33, -0.42) kontrol etti. Programın dördüncü iterasyonda durduğunu görünce de "şartı o zaman mı sağladı?" diye sordu.

> **Ders sonrası not:** Python ile hesaplandı: $\max|\Delta x| < 0.001$ ölçütü beşinci iterasyonda sağlanır. Programın saymayı sıfırdan başlatması ya da başka bir ölçüt kullanması büyük olasılıkla bu farkı açıklar. Ekrandaki çıktı altyazıdan görülmüyor.

Trapez sınaması Simpson'daki fonksiyonla yapıldı: $f(x) = 2x^5 - 3x^2 + 3$, $[1, 4]$, $n = 6$. Formül (ara adım eklendi):

$$I \approx h \left[\frac{f(x_0) + f(x_6)}{2} + \sum_{i=1}^{5} f(x_i) \right], \qquad h = 0.5$$

Sonuç 1363.59 çıktı, hoca doğru dedi.

> **Ders sonrası not:** Python ile hesaplandı: trapez değeri 1363.59375'tir. Aynı n için Simpson 1311.625, tam değer 1311'dir, yani bu fonksiyonda trapezin hatası Simpson'dan çok büyüktür.

11. Ortak Fonksiyona Parametre Aktarımı

Bir öğrencinin programında bisection ve türev dahil altı yöntem, polinom değerini hesaplayan tek bir ortak fonksiyona bağlıydı. Katsayılar giriş fonksiyonunda doğru görünüyor, hesap fonksiyonuna girince ise yalnız ilk iki değer (0 ve -5) kalıyordu. Bu yüzden bağlı yöntemlerin hepsi yanlış sonuç veriyordu.

11.1 Hocanın Teşhisi

Hoca önce sonucu okudu: "Gönderdiğin şeyleri almıyorsa parametre göndermede hatan var demektir." Modüler yapmaya çalışmanın sonucunu da şöyle özetledi: girdiler ayrı bir fonksiyonda alınıyor ama o değerler diğer fonksiyonlara gönderilemiyor.

Sonra birlikte izleme yapıldı. Hoca burada terimi de açıkladı: trace etmek, debug yapmak demektir. Girdi olarak $x^2 - 4x - 5$ verildi, izleme Newton-Raphson çağrısından başlatıldı.

Hoca çağrıdaki parametreleri ve fonksiyon tanımındakileri tek tek saydı: ikisinde de beş parametre vardı. Fakat kat sayı dizisi iki boyutluydu: bir boyutu derece ve kat sayı için 2, diğer boyutu 50 olarak tanımlanan `MAX_SIZE` idi. Hocanın sonucu şuydu: "Gönderdiğin parametrelerle aldığın parametreler eşleşmiyor. O yüzden buradaki bazı şeyler kayboluyor."

> **Ek bilgi (derste anlatılmadı):** C'de iki boyutlu dizi aktarılırken ikinci boyut, çağıran ile alan tarafta aynı olmalıdır, çünkü indis hesabı buna göre yapılır. Çağıran `double a[50][2]`, alan `double k[2][50]` olursa elemanlar farklı yerden okunur. Bu durum gcc 15 ile denendi: derleme `-Wincompatible-pointer-types` hatasıyla durdu, eski derleyiciler büyük olasılıkla yalnız uyarı verir. O uyarı kapatılınca $\{0, -5\}, \{1, -4\}, \{2, 1\}$ girdisinde `k[0]` satırı 0, -5, 1 okudu, `k[1]` satırı ise 0 geldi. Altyazıdaki belirti büyük olasılıkla bu tür bir boyut uyuşmazlığıdır, kodun tamamı altyazıda görülmüyor.

11.2 Önerilen Düzeltme

Hoca iki yol gösterdi:

- **Her yöntemin girdisini kendi içinde almak:** Sınır ve iterasyon sayısı gibi değerler gönderilir, katsayılar fonksiyonun içinde kullanıcıdan alınır. Böylece bozuk aktarım devreden çıkar.
- **Yöntemleri ortak fonksiyona bağlamadan yazmak:** Gauss-Seidel'ın, türevin ve diğerlerinin girdileri ayrı ayrı alınırsa bir hata bütün yöntemleri durdurmaz.

11.3 Çalışan Kodun Yedeği

> **Hocanın vurgusu:** "Çalışan bir kodu bozmadan evvel niye kopyasını almadın?"

Kod önceden çalışıyordu ve sonradan yapılan bir düzenlemeyle bozulmuştu. Hoca bu yüzden çalışan kodun değişiklikten önce kopyasının alınmasını istedi.

Dersin Toparlaması

Bu kayıtta görülen düzeltmeler üç başlıkta toplanır. Birincisi yöntemin gereksinimleridir: Gauss-Seidel başlangıç değeri ve uygun köşegen ister, eliminasyon başlangıç değeri istemez, Newton-Raphson aralık değil tek başlangıç değeri ister. İkincisi sınama alışkanlığıdır: kök yerine konularak, türev analitik sonuçla karşılaştırılarak kontrol edilir.

Üçüncüsü kod düzenidir: ondalıklı değerler `double` tutulur, girdi soruları net yazılır, fonksiyona gönderilen ve alınan parametreler birebir eşleşir, çalışan kodun kopyası saklanır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.